



UNIVERSIDAD PRIVADA DEL VALLE
FACULTAD DE INFORMÁTICA Y ELECTRÓNICA
CARRERA DE LICENCIATURA EN INGENIERÍA
DE SISTEMAS INFORMÁTICOS

REDES NEURONALES PARA PREDICCIÓN DE INTENCIÓN DE COMPRA

ESTUDIANTE: DAVID ANTONIO ESPADA SILVA
LUIS ALEJANDRO UGARTE LUJAN
DOCENTE: EFRAIN LUNA MAMANI



Indice

1. Introducción	3
2. Descripción del Dataset	3
2.1 Fuente e información general	3
2.2 Variables del dataset	3
2.3 Distribución de la variable objetivo	4
3. Análisis Exploratorio de Datos (EDA)	5
3.1 Estadísticas descriptivas	5
3.2 Correlaciones identificadas	5
3.3 Análisis por tipo de visitante	5
3.4 Análisis temporal	6
4. Preprocesamiento	6
4.1 Codificación de variables	6
4.2 División del conjunto de datos	6
4.3 Normalización	6
5. Fundamentos Teóricos del Modelo	6
5.1 Redes Neuronales Artificiales	6
5.2 Arquitectura implementada	7
5.3 Base matemática	7
5.4 Proceso de entrenamiento	8
5.5 Hiperparámetros clave	8
5.6 Ventajas y limitaciones	8
6. Implementación	8
6.1 Librerías utilizadas	8
6.2 Red Neuronal Principal (03_model_main.ipynb)	8
6.3 Modelo comparativo: Random Forest (04_model_comparison.ipynb)	9
7. Resultados	9
7.1 Comparativa de métricas (clase Revenue = 1)	9
7.2 Red Neuronal sin pesos de clase	9
7.3 Red Neuronal con class_weight	10
7.4 Random Forest (modelo comparativo)	10
7.5 Variables más importantes (Random Forest)	10
8. Análisis e Interpretación	11
9. Conclusiones	11
10. Referencias Bibliográficas	12



1. Introducción

El comercio electrónico ha experimentado un crecimiento exponencial en los últimos años, generando una enorme cantidad de datos sobre el comportamiento de los usuarios durante sus sesiones de navegación. Comprender si una sesión de compra finalizará en una transacción exitosa representa un desafío central para las empresas de retail en línea, ya que permite optimizar campañas de marketing, personalizar experiencias y aumentar la tasa de conversión.

El presente informe aborda este problema mediante el uso de Redes Neuronales Artificiales (RNA), un paradigma de aprendizaje automático inspirado en el funcionamiento del cerebro humano, capaz de capturar patrones no lineales complejos en los datos. Se utiliza el conjunto de datos Online Shoppers Purchasing Intention Dataset, disponible en el UCI Machine Learning Repository, que contiene información detallada sobre 12,330 sesiones de usuarios en una tienda en línea.

El objetivo principal es construir un clasificador binario que, a partir de variables de comportamiento de navegación, características de la sesión y métricas de interacción, sea capaz de predecir si una sesión culminará en una compra (variable Revenue). Se implementa además un modelo comparativo basado en Random Forest para evaluar el rendimiento relativo de la red neuronal.

2. Descripción del Dataset

2.1 Fuente e información general

El dataset Online Shoppers Purchasing Intention fue creado por Sakar et al. (2019) y puesto a disposición pública en el UCI Machine Learning Repository y Kaggle. Contiene registros de 12,330 sesiones web de una tienda de comercio electrónico durante un período de un año, diseñado de forma que cada sesión pertenece a un usuario diferente para evitar sesgos de tendencia hacia una campaña u ocasión específica.

Características principales: 12,330 filas y 18 columnas (17 variables predictoras + 1 variable objetivo).

- Clases desbalanceadas: aproximadamente 84.5% de sesiones sin compra (Revenue = False) y 15.5% con compra (Revenue = True).
- Sin valores nulos en el dataset original.
- Variables numéricas: 10 | Variables categóricas: 3 | Variables booleanas: 2

2.2 Variables del dataset

Variable	Tipo	Descripción
Administrative	Numérica	Número de páginas administrativas visitadas
Administrative_Duration	Numérica	Tiempo en páginas administrativas (seg.)
Informational	Numérica	Número de páginas informativas visitadas
Informational_Duration	Numérica	Tiempo en páginas informativas (seg.)
ProductRelated	Numérica	Número de páginas de producto visitadas
ProductRelated_Duration	Numérica	Tiempo en páginas de producto (seg.)



BounceRates	Numérica	Tasa de rebote del sitio
ExitRates	Numérica	Tasa de salida del sitio
PageValues	Numérica	Valor promedio de las páginas visitadas
SpecialDay	Numérica	Proximidad a fechas especiales (0-1)
Month	Categórica	Mes de la visita
OperatingSystems	Numérica	Sistema operativo del usuario
Browser	Numérica	Navegador utilizado
Region	Numérica	Región geográfica del usuario
TrafficType	Numérica	Tipo de tráfico web
VisitorType	Categórica	Tipo de visitante (nuevo/recurrente)
Weekend	Booleana	Si la visita ocurrió en fin de semana
Revenue	Booleana (target)	Si la sesión terminó en compra

2.3 Distribución de la variable objetivo

La variable objetivo Revenue presenta un notable desbalance de clases. Aproximadamente el 84.5% de las sesiones no resultan en compra, mientras que solo el 15.5% termina en una transacción. Este desbalance fue un factor crítico a considerar durante el entrenamiento del modelo neuronal, ya que un clasificador naíve que prediga siempre 'sin compra' podría alcanzar ~84% de accuracy sin aprender nada útil.

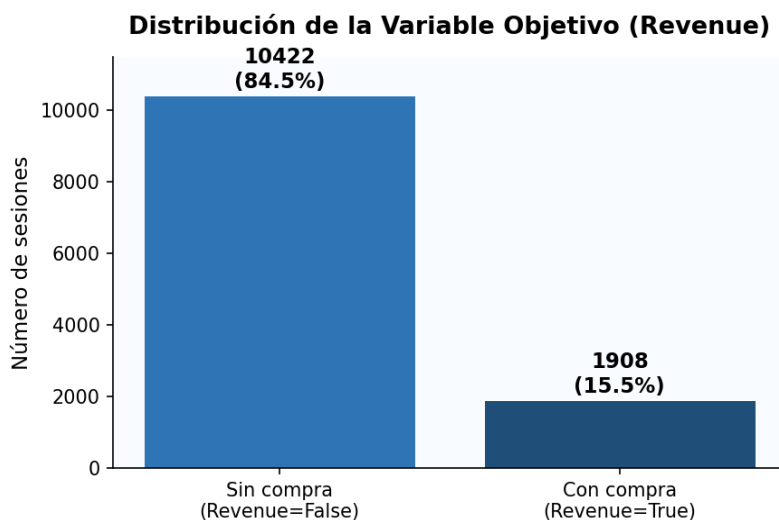


Figura 1. Distribución de la variable objetivo Revenue en el dataset.



3. Análisis Exploratorio de Datos (EDA)

3.1 Estadísticas descriptivas

El análisis exploratorio reveló varios patrones relevantes en los datos. Las variables numéricas presentan distribuciones altamente sesgadas hacia la derecha, especialmente PageValues, ProductRelated_Duration y Administrative_Duration, lo que indica que la mayoría de usuarios visitan pocas páginas por sesión, pero algunos tienen sesiones muy largas.

3.2 Correlaciones identificadas

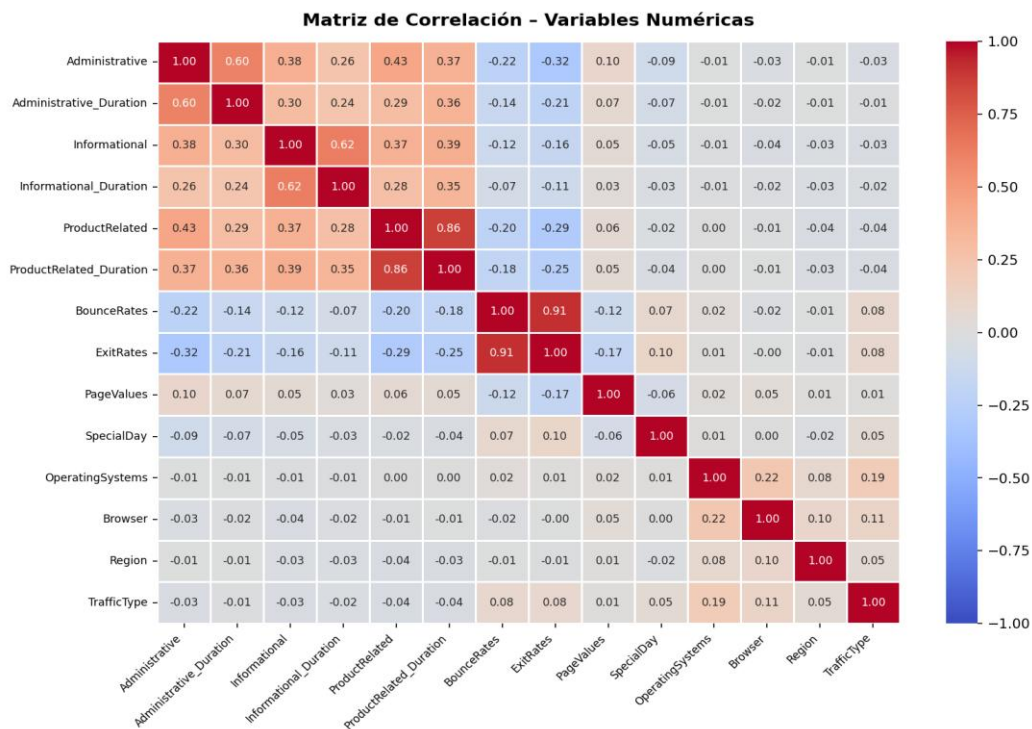


Figura 2. Matriz de correlación de las variables numéricas del dataset.

La matriz de correlación de las variables numéricas reveló relaciones importantes:

- BounceRates y ExitRates presentan una correlación positiva muy alta ($r \approx 0.91$), indicando que ambas variables describen comportamientos similares de abandono de sesión.
- ProductRelated y ProductRelated_Duration muestran alta correlación ($r \approx 0.86$), lo esperado ya que más páginas de producto implica mayor tiempo.
- Durante el análisis exploratorio, PageValues mostró una fuerte relación con el comportamiento de compra y posteriormente fue identificada por Random Forest como la variable más importante para la predicción.
- Administrative y Administrative_Duration también muestran alta correlación entre sí ($r \approx 0.60$).

3.3 Análisis por tipo de visitante

La distribución por tipo de visitante muestra que la gran mayoría de sesiones provienen de Returning Visitors (visitantes recurrentes), seguidos por New Visitors (nuevos visitantes) y una pequeña proporción de 'Other'. Los visitantes recurrentes tienen una tasa de conversión ligeramente mayor, posiblemente porque ya conocen el sitio y tienen mayor intención de compra.



3.4 Análisis temporal

El análisis por mes muestra que noviembre concentra la mayor cantidad de sesiones, coincidiendo con el Black Friday y la temporada navideña. Los meses de febrero y junio presentan los menores volúmenes de tráfico. La variable SpecialDay captura la proximidad a fechas especiales y tiende a mostrar valores altos en períodos de mayor actividad comercial.

4. Preprocesamiento

4.1 Codificación de variables

Conversión de booleanos:

Las variables Weekend y Revenue, originalmente de tipo booleano, fueron convertidas a enteros (0 y 1) para compatibilidad con los modelos de scikit-learn y TensorFlow/Keras.

One-Hot Encoding:

Las variables categóricas Month (mes de la visita) y VisitorType (tipo de visitante) fueron codificadas mediante `pd.get_dummies()` con `drop_first=True` para evitar multicolinealidad. Después del One-Hot Encoding el dataset pasó de 18 a 27 columnas (26 variables predictoras y 1 variable objetivo).

Conversión de booleanos resultantes:

Las columnas booleanas generadas por `get_dummies()` fueron explícitamente convertidas a tipo `int` para asegurar compatibilidad con `StandardScaler`.

4.2 División del conjunto de datos

El dataset fue dividido en conjuntos de entrenamiento y prueba con una proporción 80/20 mediante `train_test_split` de scikit-learn, utilizando `random_state=42` para reproducibilidad y `stratify=y` para mantener la proporción de clases en ambos subconjuntos:

- Conjunto de entrenamiento: 9,864 registros (80%)
- Conjunto de prueba: 2,466 registros (20%)

4.3 Normalización

Se aplicó `StandardScaler` de scikit-learn para estandarizar todas las variables numéricas, transformándolas a media 0 y desviación estándar 1. El scaler fue ajustado ÚNICAMENTE sobre los datos de entrenamiento (`fit_transform`) y aplicado a los datos de prueba (`transform`) para evitar data leakage. Los datos procesados fueron guardados en la carpeta `data/processed/` para ser cargados por los notebooks de modelado.

5. Fundamentos Teóricos del Modelo

5.1 Redes Neuronales Artificiales

Una Red Neuronal Artificial (RNA) es un modelo computacional inspirado en la estructura y funcionamiento del sistema nervioso biológico. Está compuesto por nodos (neuronas artificiales) organizados en capas que procesan información de manera paralela y distribuida. Las RNA son aproximadores universales de funciones (Teorema de Aproximación Universal, Cybenko 1989), capaces de modelar relaciones altamente no lineales entre variables.



5.2 Arquitectura implementada

Se implementó una red neuronal de tipo feedforward (alimentación hacia adelante) con la siguiente arquitectura:

- Capa de entrada: 26 neuronas (una por variable predictora tras el preprocesamiento)
- Primera capa oculta: 64 neuronas con activación ReLU
- Segunda capa oculta: 32 neuronas con activación ReLU
- Capa de salida: 1 neurona con activación Sigmoid (clasificación binaria)

Arquitectura de la Red Neuronal

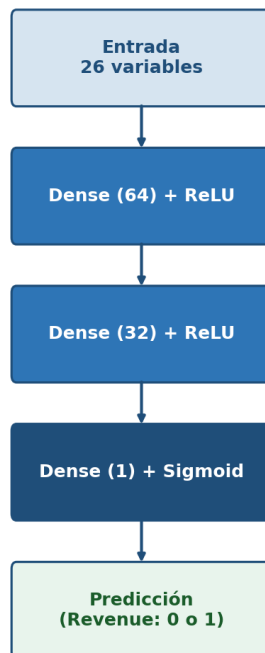


Figura 3. Diagrama de la arquitectura de la Red Neuronal implementada.

5.3 Base matemática

Función de activación ReLU (Rectified Linear Unit):

$$f(x) = \max(0, x)$$

ReLU es la función de activación más utilizada en capas ocultas. Introduce no linealidad sin saturar el gradiente para valores positivos, lo que acelera el entrenamiento y mitiga el problema de gradientes que desaparecen (vanishing gradient).

Función de activación Sigmoid:

$$\sigma(z) = 1 / (1 + e^{(-z)})$$

Utilizada en la capa de salida para problemas de clasificación binaria. Mapea cualquier valor real al rango (0, 1), interpretable como probabilidad de pertenencia a la clase positiva (Revenue = 1).

Función de pérdida – Binary Cross-Entropy:

$$L = -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$$

Donde y es la etiqueta real (0 o 1) e $\hat{y}$ es la probabilidad predicha por la red. Esta función penaliza fuertemente las predicciones muy erróneas y es la función estándar para clasificación binaria.



Optimizador Adam:

Se utilizó el optimizador Adam (Adaptive Moment Estimation), que combina las ventajas de RMSprop y Momentum. Adam adapta la tasa de aprendizaje para cada parámetro individualmente, lo que lo hace robusto y eficiente para la mayoría de problemas de deep learning.

5.4 Proceso de entrenamiento

El proceso de entrenamiento consiste en la propagación hacia adelante (forward pass) para calcular las predicciones, el cálculo de la función de pérdida, la propagación hacia atrás (backpropagation) para calcular los gradientes mediante la regla de la cadena, y la actualización de los pesos mediante el optimizador. Este ciclo se repite durante 30 épocas con mini-batches de 32 muestras.

5.5 Hiperparámetros clave

- Épocas: 30 – número de veces que el modelo ve el conjunto de entrenamiento completo.
- Batch size: 32 – número de muestras procesadas antes de actualizar los pesos.
- Learning rate: 0.001 (valor por defecto de Adam) – velocidad de aprendizaje.
- Validation split: 0.2 – 20% de los datos de entrenamiento usados para validación en tiempo real.
- Class weights: balanceados para compensar el desbalance de clases en la segunda configuración.

5.6 Ventajas y limitaciones

Ventajas:

- Capacidad de aprender representaciones no lineales complejas.
- Escalabilidad a grandes volúmenes de datos.
- Flexibilidad arquitectónica: permite ajustar capas, neuronas y funciones de activación.
- Buen rendimiento en problemas de clasificación tabular con preprocesamiento adecuado.

Limitaciones:

- Requiere mayor cantidad de datos para generalizar bien comparado con modelos clásicos.
- Menor interpretabilidad (caja negra) frente a modelos como árboles de decisión.
- Sensible a la escala de los datos (requiere normalización).
- Puede sobreajustarse si no se aplica regularización (Dropout, L2) en datasets pequeños.

6. Implementación

6.1 Librerías utilizadas

- TensorFlow / Keras: construcción y entrenamiento de la red neuronal.
- scikit-learn: preprocesamiento (StandardScaler, train_test_split), métricas y Random Forest.
- pandas / numpy: manipulación y procesamiento de datos.
- matplotlib / seaborn: visualizaciones estadísticas.

6.2 Red Neuronal Principal (03_model_main.ipynb)

Se construyó el modelo usando la API Sequential de Keras con tres capas Dense. La primera configuración (sin pesos de clase) mostró un accuracy de 88.16%, pero un recall de solo 0.55 para



la clase positiva (Revenue=1), confirmando el efecto del desbalance de clases: el modelo tiende a predecir la clase mayoritaria.

En la segunda configuración se aplicó `compute_class_weight` con `class_weight='balanced'` de scikit-learn para calcular pesos inversamente proporcionales a la frecuencia de cada clase. Los pesos se pasaron al método `model.fit()` como parámetro `class_weight`. Esta estrategia mejoró el recall a 0.73, a costa de una reducción en accuracy (84.63%) y precisión (0.50). Este trade-off es deseable en el contexto del negocio: identificar más sesiones que sí terminan en compra es más valioso que mantener un alto accuracy global.

6.3 Modelo comparativo: Random Forest (04_model_comparison.ipynb)

Se implementó un `RandomForestClassifier` de scikit-learn con los siguientes hiperparámetros:

- `n_estimators=200`: 200 árboles de decisión en el conjunto.
- `max_depth=10`: profundidad máxima de cada árbol para evitar sobreajuste.
- `random_state=42`: semilla para reproducibilidad.
- `class_weight='balanced'`: mismo tratamiento del desbalance de clases que la RNA.

El Random Forest permite además extraer la importancia de las variables (feature importances), proporcionando una visión interpretable de cuáles variables son más predictivas de la intención de compra.

7. Resultados

7.1 Comparativa de métricas (clase Revenue = 1)

Modelo			Accuracy	Precision	Recall	F1-Score
Red Neuronal	(sin	class_weight)	88.16%	0.63	0.55	0.59
Red Neuronal	(con	class_weight)	84.63%	0.50	0.73	0.60
Random Forest (comparativo)			87.35%	0.57	0.76	0.65

Nota: Los valores de Precision, Recall y F1-Score corresponden a la clase positiva (Revenue = 1), que es la clase de interés para la predicción de intención de compra.

7.2 Red Neuronal sin pesos de clase

El primer modelo neuronal alcanzó un accuracy de 88.16%. Sin embargo, el recall para la clase positiva fue de apenas 0.55, lo que significa que el modelo no detectó el 45% de las sesiones que realmente terminaron en compra. Este comportamiento es típico en problemas con desbalance de clases: el modelo aprende a predecir la clase mayoritaria (sin compra) la mayor parte del tiempo.



7.3 Red Neuronal con class_weight

Al aplicar pesos de clase balanceados, el recall de la clase positiva mejoró a 0.73, a costa de reducir el accuracy a 84.63% y la precisión a 0.50. El F1-Score se mantuvo similar (0.60), pero el modelo ahora identifica una proporción mayor de compradores reales, lo cual es más útil para el negocio.

7.4 Random Forest (modelo comparativo)

El modelo Random Forest con 200 árboles obtuvo mejores métricas en este conjunto de datos con un accuracy de 87.35%, precisión de 0.57, recall de 0.76 y F1-score de 0.65. Sin embargo, la Red Neuronal constituye el modelo principal del proyecto debido a que el objetivo académico era aplicar técnicas de Redes Neuronales; el Random Forest se incluye únicamente como referencia comparativa.

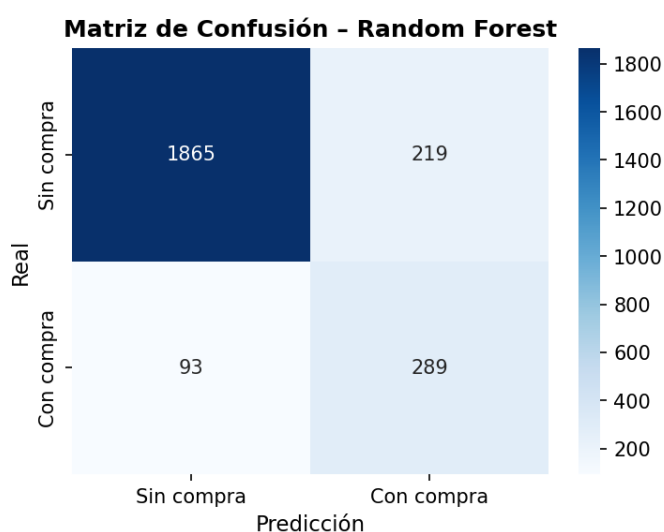


Figura 4. Matriz de confusión del modelo Random Forest sobre el conjunto de prueba.

7.5 Variables más importantes (Random Forest)

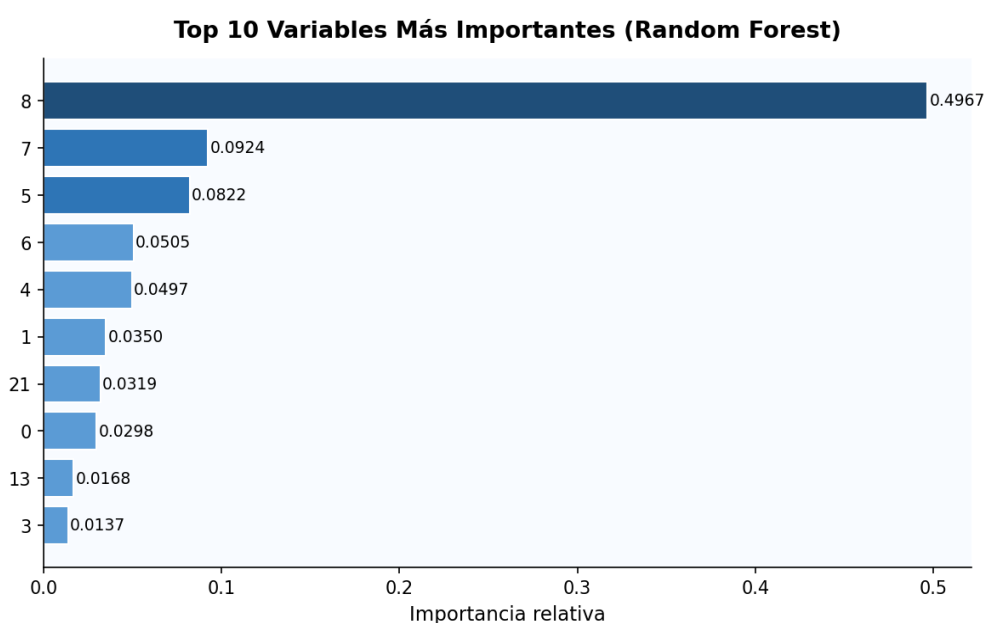


Figura 5. Top 10 variables más importantes según el modelo Random Forest.



PageValues resultó ser la variable más importante con amplia diferencia. Durante el análisis exploratorio, PageValues mostró una fuerte relación con el comportamiento de compra y fue confirmada por Random Forest como la variable más importante para la predicción. Le siguen en importancia ExitRates, ProductRelated_Duration y BounceRates.

8. Análisis e Interpretación

Los resultados obtenidos ofrecen varias conclusiones relevantes tanto desde la perspectiva técnica como de negocio:

Sobre el desbalance de clases:

El desbalance (85/15) es el principal desafío del dataset. La estrategia de class_weight fue efectiva para mejorar la detección de la clase positiva. En aplicaciones reales podrían complementarse con SMOTE (oversampling sintético) o umbral de decisión adaptativo.

Sobre PageValues como predictor dominante:

La alta importancia de PageValues confirma que los usuarios que se acercan al proceso de checkout (páginas de alto valor) tienen una probabilidad significativamente mayor de completar la compra. Esta variable es calculada por el propio sistema de analytics y representa el valor económico de cada página en el camino a la conversión.

Sobre el trade-off Precision-Recall:

La configuración con class_weight reduce la precisión (más falsos positivos) pero aumenta el recall (menos falsos negativos). En el contexto de e-commerce, los falsos negativos son más costosos: perder una sesión que iba a terminar en compra es peor que activar una oferta innecesaria. Por esto, la RNA con class_weight es la configuración recomendada como modelo principal.

Comparación RNA vs Random Forest:

El Random Forest obtuvo mejores métricas en este conjunto de datos; sin embargo, la Red Neuronal constituye el modelo principal del proyecto debido a que el objetivo académico era aplicar técnicas de Redes Neuronales. Este resultado es consistente con la literatura: los modelos basados en árboles suelen rendir mejor que las redes neuronales en datasets tabulares de tamaño mediano con variables numéricas y categóricas mixtas.

9. Conclusiones

El presente trabajo demostró que es posible predecir la intención de compra de usuarios en una tienda en línea con alto grado de efectividad mediante técnicas de machine learning. Los principales aprendizajes obtenidos son:

1. Las redes neuronales son un modelo poderoso para clasificación binaria, especialmente cuando se configuran correctamente para manejar el desbalance de clases mediante pesos balanceados.
2. El preprocesamiento es fundamental: la normalización con StandardScaler y la codificación adecuada de variables categóricas tuvieron impacto directo en la capacidad del modelo de converger y generalizar.
3. PageValues resultó ser la variable más predictiva, confirmada por el análisis de importancia del Random Forest, lo que sugiere que el comportamiento del usuario en páginas cercanas al checkout es el indicador más fuerte de intención de compra real.



4. El Random Forest obtuvo mejores métricas en este conjunto de datos; sin embargo, la Red Neuronal constituye el modelo principal del proyecto debido a que el objetivo académico era aplicar técnicas de Redes Neuronales.
5. La métrica de evaluación debe alinearse con el objetivo de negocio: en este caso, el recall de la clase positiva es más informativo que el accuracy global debido al desbalance de clases.

Limitaciones y posibles mejoras:

- Implementar técnicas de oversampling como SMOTE para abordar el desbalance de forma más robusta.
- Explorar arquitecturas más profundas con regularización (Dropout, BatchNormalization).
- Realizar búsqueda de hiperparámetros (GridSearchCV, Bayesian Optimization) para optimizar la configuración del modelo.
- Evaluar modelos de boosting (XGBoost, LightGBM) como alternativas al Random Forest.
- Incorporar validación cruzada estratificada para estimaciones más robustas del rendimiento.

10. Referencias Bibliográficas

Sakar, C. O., Polat, S. O., Katircioglu, M., & Kastro, Y. (2019). Real-time prediction of online shoppers' purchasing intention using multilayer perceptron and LSTM recurrent neural networks. *Neural Computing and Applications*, 31(10), 6893–6908. <https://doi.org/10.1007/s00521-018-3523-0>

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. <http://www.deeplearningbook.org>

Chollet, F. (2021). *Deep Learning with Python* (2nd ed.). Manning Publications.

Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.

UCI Machine Learning Repository. (2018). *Online Shoppers Purchasing Intention Dataset*. <https://archive.ics.uci.edu/ml/datasets/Online+Shoppers+Purchasing+Intention+Dataset>